The Phrozen Keep - Knowledge Base

Written by: Nefarius
Written on: Thu Sep 07, 2006 11:21 am
Last modified: Mon Oct 22, 2018 12:37 am

In this post I´m going to touch one of the areas which is usually giving mapmakers the gripes when they even think about it: AutoMaps. When messing with maps in Diablo2, there are a few instances where we will run into problems with the automap, for example when adding .dt1s to an existing level type in LvlTypes.txt or when importing new tiles into the game. In such cases, the automap may not show up at all, or it will have gaps and missing parts. In other instances it might show up wrong elements (e.g. when you imported a new statue, it shows up as a wall part or a tree on the automap).

To fix problems like those, we´ll have to make some changes in AutoMap.txt and while this file with its ~3300 rows full of numbers and odd codes looks rather freaky at first sight, it is pretty easy to handle once you know about it. After some tests, I was able to make a new custom automap for the demon crypt levels in onyx´ mod Back to Hellfire in less than half an hour, so adding new automaps isn´t necessarily the tremendous, time-consuming Uber-task that it seems to be.

Before we actually start with the detailed How to, I´d like to give a short overview of how automaps work at all and what role automap.txt plays in the whole process. After that, we´ll have a closer look into the file itself and learn what the individual columns are for (aka fileguide).

And finally, we´ll look at a few examples of how to create a new automap for a level. One of them will show us a very fast, but yet efficient way to setup new automaps very easily, another example will introduce how to convert automaps from other leveltypes and I´ll also give some tips how to set up sophisticated automaps like in the default game.

The Automap - How it works

Alright, you know, when walking around in the game the automap gets successively extended. If you walk through a dungeon, it will schematically display walls, doors, warps and such stuff in your proximity and will permanently store the maps of areas you discovered already. An exception to this are the town maps, which always get drawn completely when you enter them the first time. Additionally, town automaps in act2, 4 and 5 have a speciality which we´ll have a look at below.

First, let´s find out where the automap graphics are stored. Go to the following directory in the d2exp.mpq :

data\global\ui\Automap
In this directory, you´ll find two .dc6 files, MaxiMap.dc6 (for the standard automap) and MaxiMapS.dc6 (for the smaller minimap introduced in LoD). Both of them are multi-frame .dc6 files which contain all the graphics for the automaps in form of little 'icons', which can be up to 16x32 (B x H) pixels large (8x16 for the minimap). You can view all these icons using DrTester, however, simply for practical reasons it is better to use this chart, which was created with Paul Siramy´s Dc6Table Tool. After all, these two .dc6s contain roughly 1500 different icons each and if you later want to choose good ones for your custom automaps, browsing through them with DrTester would be... umm... inconvenient .

You´ll also notice a couple of other files in data\global\ui\Automap , namely the following ones:

```
Act2Map.dc6 (located in d2data.mpq)

Act2MapS.dc6

Act4Map.dc6 (located in d2data.mpq)

Act4MapS.dc6

ExTnMap.dc6

ExTnMapS.dc6
```

These are the special automap files for the act2, 4 and 5 towns. In contrast to the normal areas, they are not composed of many little icons, instead they use larger chunks of automap graphics which are put together, so when you rearrange towns and want your changes reflected in the automaps, you´ll have to deal with these files.

Ok, now that we know where the automap graphics come from, how does the game know where to draw these 'icons'? This is actually closely linked to the tiles that the map is composed of. Each individual tile can have an automap icon assigned to it, so that when the players gets near the tile, the respective icon will be displayed in the automap. Let´s, for example, assume that the players is walking past a cottage in act1. The wall tiles of the cottage then have automap icons assigned to them, which, when composed together, will display a house so when we walk by the cottage tiles, they will let the relevant icons appear on the automap. The allocations that tell the game which tile will display which icon are made in a .txt file and this .txt file (you guess it) is nothing but our automap.txt.

Not all tiles have automap graphics assigned to them, usually only distinct objects or level parts have. In the act1 wilderness, for example, most parts of the automap remain blank and only the paths and level borders are drawn.

Automap.txt - Fileguide

Alright, we now know that automap.txt is responsible for assigning map icons to individual tiles, so let´s have a look at how it does that. Basically, we can divide all the columns in there into two groups: the first few define a tile or a range of tiles and the others then determines an icon which is used for this/ these tiles.

LevelName: The entry here is composed of the number of the act (1-5) and the name of the level type.
Level types seem to be hardcoded, so you cannot simply add entries for a new level type you created in LvlTypes.txt. Inserting new lines for an existing level type however is possible.

TileName: This column contains codes, which actually refer to a certain tile orientation. We are lucky here, because SVR has recently figured out which one is which one:

TileName	Orientation	*desc
fl	0	floor
wl	1	wall left
wr	2	wall right
wtlr	3	wall top lower right
wtll	4	wall top lower left
wtr	5	wall top right
wbl	6	wall bottom left
wbr	7	wall bottom right
wld	8	wall left door
wrd	9	wall right door
wle	10	wall left exit
wre	11	wall right exit
co	12	center object
sh	13	shadow
tr	14	tree
rf	15	roof
ld	16	left down
rd	17	right down
fd	18	full down
fi	19	full invisible?? (paired with 18)

Style: Style is the same as the tile Main Index, the numbers are decimal.

StartSequence / EndSequence: Sequence is the same as the tile Sub Index and with Start and End you can define a range of indices.
For example, a **StartSequence** of 4 and **End Sequence** of 8 will address tiles with Sub Index 4, 5, 6, 7 and 8. Again, all numbers are decimal.
In some rows you can find a value of -1; this means that the Sub Index is not important and the game will only look at Style and TileName to determine which tiles are addressed.

All four columns together give a unique definition of what tiles are addressed in a certain row. LevelName limits the range of possible tiles into a certain level type of a distinct act. TileName, Style and Sequence (more commonly refered to as Orientation, Main Index and Sub Index) then exactly define the tile(s) that are addressed, since there usually is only one tile with a certain combination of the three values for the given level type. The Start and End columns for the Sub Index allow to adress more than one tile in a single row, which safes us some space when we have several variants of a single tile, but don´t want them to show up differently on the automap (e.g. variations of walls).

The next few columns define the exact automap graphic that will be used for the specified tile(s).

Type1-4: These are just comment fields, as far as I know. Put in whatever you like, but you will probably want to use something descriptive so you know what tiles this line deals with.

Cel1-4: This column contains a number which determines the frame of the MaxiMap(s).dc6 that will be applied to the specified tiles.
Figuring those out is probably the most tendious part when creating new automaps. Refer back to this image. Here, each line holds 20 images (when you re-extract the chart with Dc6Table, you can specify how many graphics a line can hold), so line 1 includes icons 0-19, line 2 from 20 to 39 and so on. Enter the number of the respective icon and it will show up for the specified tile(s).

You´ve noticed, there are four Type columns and analogous four Cel columns. Why that? Well, it serves to give our automap a bit randomness and variation. If there is more than just one entry in these columns (e.g. if you put something in 1 and 2), the game will randomly pick one of them if a tile is to be displayed in the automap. This is especially usefull if you have large areas with essentially the same tiles in them. Using the same icon for all of them over and over again might look dull or would introduce patterns and such. By randomly using different icons, it will all look natural and won´t have any repeating patterns on it.

That pretty much sums it up for an overview of Automap.txt, so let´s next have a look at how to actually make new automaps.

Building your own automaps

Now let´s get serious and look at the way how to set up automaps. I´ll explain some methods and tricks that I found usefull while analyzing AutoMap.txt and setting up the automaps for the dungeon in onyx´mod. One method will put out only a rudimentary automap that will just show you the general shape of the dungeon/level and some important spots. This one however is very quick and doesn´t involve much work, so it´s ideal if you´re in a hurry with your project and don´t want to spend too much time on the automap. Next you might want to know how to fix the automap if you added .dt1s from other level types and got weird results in the automap. Finally, we will discuss some efficient techniques of how to set up complex automaps with many details and have a look at some problems that could occur there.

Alright then, for the first method it is really simple. Basically all maps in D2 have floor tiles as some sort of 'underground' for the player to walk on (some of them might be unwalkable due to the floor flag settings, but we won´t consider that here). Try the following: Open up a larger map in WinDs1edit, toggle off all wall layers and zoom out. You won´t see any finer structures of the level now, but you have a very good overview of the general layout. This is similar to what we could do with the automap: We could simply tell it to display all floor tiles, so that we would get something like a 'floorplan' of the dungeon. Of course, this method will have its flaws, since from that 'floorplan' we can´t tell if there are dead ends in some parts of the dungeon and we´d still have to use different automap symbols for special stuff like stairs or warps. However, this method works nicely for a rough overview (check this example from a new map in onyx´mod Back to Hellfire) and involves little work in Automap.txt. Basically, all you have to do is look up all the indexes that are used for the floor tiles, open up Automap.txt, insert a couple of new lines under the level type you´re using for your new level and fill out the values appropriately. Simply use one and the same automap graphic for all tiles and you´ll end up with something like in the image above. For special things like warps, you have to figure out what floor tiles they use and assign them a different icon, so they become visible on the automap.

To import new maps into D2, Joel´s and Paul´s TileMaker tool is very convenient and since it only creates floor tiles, it seems to be determined for the abovementioned method. The tool however has a little problem that we need to consider. Look at this sample image and what the TileMaker tool made out of it. You´ll notice that there are lots of tiles which are completely empty (pure black, no level elements on them). Since TileMaker converts the whole image into tiles, you´ll always have lots of such tiles from the borders or corners of your source image. The point now is: If you assign automap graphics to these tiles too, you´ll get an indifferent plane as your automap, because the parts of the level that are actually empty are also drawn in your minimap. There are several ways to fix this problem, but only one is really practical. You could delete all the empty tiles from the .dt1, but that is tedious and involves alot of looking up and changing the .ini file. Same goes for deleting all references to such tiles in Automap.txt, for bigger maps you´d have to look up many indexes which is just painfull. The easiest way is to go into the .ds1 file that TileMaker has created and delete all empty tiles from the map. Since they are not there anymore, only the tiles that actually have 'something' on them will show up on the automap.

*You might run into problems, if the background colour of you source image for the imported map is not pure black like the default background in D2 - make sure it is, otherwise your tiles will be distiguishable from the background, which looks bad.

Now let´s consider a different case. Let´s say you expanded one of the caves in act1 with a few special rooms, maybe an endboss lair or whatever, and for those you used tiles from the barracks (basewall.dt1). The level plays fine, but the automap doesn´t show the new tiles from the barracks. Why does this happen?

Well, obviously, automap.txt must be missing the entries for the new tiles, how in the world would it know we added new ones? You can easily check this out yourself, the barrack tiles all use main index 5, while the caves have entries in automap.txt for main indexes 0, 1, 2, 3, 4 and 21 only. On a sidenote, what would have happened if the barrack tiles´ indexes would have matched the entries in automap.txt? It´s obvious again: If the barracks tiles´indexes would have matched to the entries in the file, they´d get automap graphics for cave elements, since that´s what is defined there.

To fix the problem, we can simply go and find the rows for these tiles among the original '1 Barracks' (lines 277 - 324) section in automap.txt, copy them and then add them under the default lines in the '1 Cave' (lines 426 - 551) section. Use Orientation, Main Index and Sub index to identify the right lines (Black Heart´s Warped Tiles site is pretty handy for this job).

Let´s get to the last point, how to set up a complex automap for a completely new area or dungeon? It pretty much boils down to assigning the right icons to the right tiles, but there are some things which are worth to spend some attention on:

Firstly, try to orientate on Blizzard´s style when builing the automap. Use icons for either the walls or the floors, mixing things will quickly overload the automap and look confusing. Check existing maps to find out a 'set' of automap graphics (like I did here for onyx´ new dungeon). Also keep in mind to use suitable colours for the automap, a grey automap in a mostly grey dungeon will make it difficult for the player to see things. For warps or stairs, use icons with outstanding colours, so they can be easily spotted when viewing the automap.

*On a sidenote, it is possible to extend the Automap.dc6 file with custom graphics if you need to. Just add your new icons in the appropriate size at the end of the table created with Dc6Table and when recompiling it, the tool will automatically import them as new frames, which can be adressed by the Cel values.

Building the automap is usually not problematic, there is however a little thing that I want to point out here, so you don´t get baffle if it occurs in your projects and you can´t get rid of it. Sometimes we have the case that there are several tiles on the same tile but in different layers. If each of these stacked tiles has a corresponding entry in automap.txt, we´re in trouble, because unlike tiles, the automap icons cannot stack. Instead, the game will choose the icon of the tile with the higher orientation (e.g. orientation 7 overrides orientation 1) and display only that. This can lead to small gaps and holes in your automap in those spots where tiles are stacked in different layers.

There is no real way around this problem, unless we are clever enough to take this fact into consideration before we start cutting the tiles. However, the effect is hardly noticable and does not disturb the functionality of the automap. For the original D2 tiles, they were created in such a way that the automap icons nearly always match up correctly, so we won´t have problems with them.

That´s all for it so far, hope it did clear you up on some things concerning automaps. For further discussion of questions, feel free to post here.

Link to this article: `https://d2mods.info/forum/kb/viewarticle?a=419`